

TP1 - Représentation de graphes

Dans ce TP, il est demandé de créer quelques classes en Java permettant de représenter un graphe sous forme de liste d'adjacence et d'implémenter l'algorithme du plus court chemin.

Les objectifs sont :

- de s'initier, avec Maven, aux dépendances, à la compilation et aux tests dans le cadre d'un projet structuré.
- de créer quelques classes
- de redéfinir quelques méthodes héritées de la classe `Object`
- de manipuler les conteneurs (listes, tableaux associatifs, etc.) en java.

Étape 1 : Création et configuration du projet Maven

1.1 initialisation du projet à partir d'un archetype

1. Ouvrez votre terminal ou votre invite de commande.
2. Utilisez la commande `mvn archetype:generate` pour générer un projet de base. C'est la manière la plus simple de démarrer.
 - `groupId` : L'identifiant de votre groupe. Utilisez une convention de nom de domaine inversée (ex. `fr.uga.im2ag.m1info`).
 - `artifactId` : Le nom de votre projet (ex. `tp1-graphes`).
 - `interactiveMode=false` : Permet de créer le projet directement sans questions additionnelles.

```
mvn archetype:generate -DgroupId=fr.uga.im2ag.m1info.tp1 -  
DartifactId=tp1-graphs -Dversion=1.0-SNAPSHOT -  
DinteractiveMode=false -DarchetypeVersion=1.5 -DjunitVersion=5.13.4
```

3. Examiner le répertoire `tp1-graphes` de votre nouveau projet. L'arborescence standard de Maven est créée :
 - `src/main/java/...` : Vos classes Java pour la logique de l'application.
 - `src/test/java/...` : Vos classes pour les tests unitaires.
 - `pom.xml` : Le **Project Object Model**. C'est le fichier décrivant le projet et ses dépendances vers d'autres librairies.

1.2 : Analyse rapide du fichier pom.xml

Ouvrez le fichier pom.xml. Vous y trouverez les identifiants de votre projet (groupId, artifactId, version), ainsi que la version du compilateur utilisées (<maven.compiler.release>17</maven.compiler.release>), etc. C'est également dans ce fichier que l'on déclare les dépendances vers des bibliothèques externes. Vous verrez plusieurs déclarations de bibliothèques dans l'élément <dependencies> dont **Junit 5** qui sera utilisée pour les tests unitaires :

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <scope>test</scope>
</dependency>
```

En général, lorsque vous voulez ajouter une dépendance, il faut aller la chercher sur le site catalogue [Maven Repository](https://mvnrepository.com) et copier le bloc XML de la dépendance à l'intérieur de la balise <dependencies> (à créer si elle n'existe pas).

Étape 2 : Structure de base du code

Dans le package fr.uga.im2ag.m1info.tp1, créez les quatre classes suivantes :

- Vertex : pour représenter un sommet.
- Edge : pour représenter une arête (orientée) pondérée entre deux sommets.
- Graph : pour gérer l'ensemble des sommets et de leurs connexions.
- App : le point d'entrée pour tester notre implémentation.

Pour l'ensemble des classes, respectez les principes d'encapsulation : attributs privés, seules les méthodes demandées sont publiques.

2.1 : La classe Vertex

Une instance de Vertex contient seulement le label, identifiant, du sommet. Un sommet sera initialisé en donnant son label, qui ne peut être changé par la suite. Il sera possible de récupérer le label d'un sommet via la redéfinition de la méthode de signature public String toString() héritée de la classe Object.

Ajoutez la classe de test VertexTest ci-dessous dans le répertoire src/test/java/fr/uga/im2ag/m1info/tp1 de votre projet :

```
package fr.uga.im2ag.m1info.tp1;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class VertexTest {

    @Test
    public void testVertexCreation() {
        Vertex v = new Vertex("A");
        assertEquals("A", v.toString());
    }
}
```

```

@Test
public void testVertexEquality() {
    Vertex v1 = new Vertex("A");
    Vertex v2 = new Vertex("A");
    Vertex v3 = new Vertex("B");

    assertEquals(v1, v2);
    assertEquals(v1, v3);
}
}

```

Testez votre classe avec la commande suivante :

```
mvn clean install
```

Qu'observe-t-on ? Pourquoi le test `testVertexEquality` ne fonctionne pas ? Quelle(s) méthode(s) de la classe `java.lang.Object` doit-on redéfinir pour que le test passe ? Faites le nécessaire pour que le test passe.

2.2 : La classe Edge

Les instances de `Edge` contiennent les sommets source et cible de l'arête orientée ainsi que son poids sous la forme d'un entier. Une arête est immuable (on ne peut pas modifier ses attributs mais on peut y accéder). Il est aussi possible d'obtenir une représentation de l'arête sous la forme de chaîne de caractères suivante : `ID_SOURCE -> ID_CIBLE [label=P0IDS]`. Pour cela, il est demandé de redéfinir la méthode de signature `public String toString()` héritée de la classe `Object`.

Ajoutez la classe de test `EdgeTest` et testez.

2.3 : La classe Graph

La classe `Graph` maintient l'association entre les sommets et la liste des arêtes dont ils sont la source. Pour cela, on utilisera les types `java.util.Map` et `java.util.List` de `java`. Les types `Map` et `List` sont des types génériques. `Map` est paramétrable par le type des clés et le type des valeurs, et `List` est paramétrable par le type des éléments. Dans notre cas, nous utiliserons : `Map<Vertex, List<Edge>>` (clés de type `Vertex` et valeurs de type `List` de `Edge`).

A l'initialisation, le graphe ne contient aucun sommet (et aucune arête). Le type `java.util.Map` étant abstrait (c'est une interface), vous utiliserez l'implémentation `java.util.HashMap`.

La classe `Graph` fournit les fonctionnalités pour ajouter un sommet à partir d'une instance de `Vertex` et pour ajouter une arête à partir de deux instances de `Vertex` et d'un entier qui est le poids de l'arête. Cette dernière méthode permettra d'ajouter les sommets si ils ne sont pas déjà présents dans le tableau associatif. Lors de l'ajout d'un sommet dans le tableau associatif, il faut l'associer avec une nouvelle liste (vide). Le type `java.util.List` étant abstrait (c'est une interface), vous utiliserez l'implémentation de liste `java.util.ArrayList`.

La classe `Graph` proposera également des méthodes pour obtenir la liste des arêtes associées à un sommet et la liste des sommets du graphe.

Pour la réalisation des fonctionnalités, il est conseillé d'utiliser les méthodes appropriées des types `java.util.Map` et `java.util.List`.

Finalement, réalisez une méthode `void printGraph(PrintStream out)` permettant d'afficher un graphe dans le format DOT. Voici un exemple d'affichage d'un graphe avec 3 sommets et une arête de A vers B avec le poids 3 :

```
digraph {
    A -> B [ label=3 ];
    C;
}
```

Ajoutez la classe de test GraphTest et testez. Ajoutez un test qui ajoute dans un graphe deux sommets distincts ayant le même label et qui vérifie qu'une seule des deux instances est effectivement ajoutée au graphe. Qu'observez-vous ? Si ce test ne passe pas, vous n'avez probablement pas redéfini la méthode public int hashCode() de la classe `Object` qui est utilisée par la table de hachage (`HashMap`).

2.4 : La classe App

Complétez la classe `App` pour créer un exemple de graphe avec au moins trois sommets et un arc. Affichez ce graphe sur la sortie standard (objet `System.out` de type `PrintStream`).

Pour compiler et exécuter l'application, il faut effectuer les étapes suivantes :

1. Placez-vous à la racine de votre projet dans le terminal.
2. Compilez le code avec la commande suivante (compilation, tests, packaging).

```
mvn clean install
```

3. Exécutez le programme principal avec le plugin `exec`.

```
mvn exec:java -
Dexec.mainClass="fr.uga.im2ag.m1info.tp1.App" -q
```

Vous pourrez tester la sortie de votre programme en la redirigeant dans un fichier puis en utilisant le programme `dot` de `GraphViz` pour générer une image de votre graphe :

```
mvn exec:java -
Dexec.mainClass="fr.uga.im2ag.m1info.tp1.App" -q > mon_graphe.dot
dot -Tpng mon_graphe.dot > mon_graphe.png
```

Étape 3 : Le plus court chemin vers la fin de ce tp

3.1 : La classe Path

Créer la classe `Path` qui représente un chemin par la liste des sommets composant ce chemin et le poids total du chemin. Les instances de cette classe pourront être initialisées de deux manières :

- on ne donne qu'un sommet : le chemin sera composé de ce sommet unique et son poids total sera 0
- on donne en argument un chemin et une arête : la liste des sommets du chemin initialisé sera le résultat de la concaténation des sommets du chemin et la cible de l'arête passés en arguments.

Le poids du chemin nouvellement créé sera la somme du poids du chemin et de l'arête passés en argument. Si la source de l'arête n'est pas égale au dernier sommet du chemin initial alors une exception de type `java.lang.RuntimeException` sera levée (via `throw new RuntimeException("un message expliquant le problème");`).

La classe `Path` fournira les fonctionnalités pour donner la liste des sommets du chemin ainsi que le poids total du chemin. Elle permettra également de fournir une représentation sous forme de chaîne de caractères du chemin.

Créer une classe de test permettant de vérifier les différentes façons de créer un chemin. Pour tester la levée d'une `RuntimeException`, vous pouvez utiliser la syntaxe suivante (qui utilise la notation fonctionnelle de java) :

```
assertThrows(RuntimeException.class, () -> new Path(begin,next));
```

Un chemin est-il réellement immuable ? Peut-on, par exemple, ajouter un sommet sur la liste des sommets d'un chemin (testez). Quel problème pose cette possibilité d'ajouter un sommet directement sur les sommets d'un chemin ? Quelle solution proposez-vous pour éviter qu'un ajout soit possible ? Une des possibilités est de retourner une vue non modifiable (aller voir dans la classe utilitaire `java.util.Collections`). Proposer une méthode de test qui vérifie que l'on ne puisse pas modifier la liste des sommets.

Pour la suite, il sera nécessaire de comparer des chemins selon leur poids. Pour cela, il faut rendre les instances de `Path` comparables, en suivant les étapes suivantes :

- dire que la classe implémente l'interface `java.lang.Comparable` : `public class Path implements Comparable<Path>`
- ajouter la méthode de signature `public int compareTo(Path p)` définie par `Comparable`.

Cette méthode représente une relation d'ordre totale et `p1.compareTo(p2)` retourne 0 si et seulement si `p1` est égal à `p2`. Ainsi dans le cas où deux chemins ont le même poids, on comparera leur nombre de sommets, puis si ils ont le même nombre de sommets, on comparera les noms des sommets (les instances de `String` sont comparables).

Ne pas oublier d'ajouter une (ou plusieurs) méthodes de test pour vérifier votre implémentation.

3.2 : La classe `DijkstraSolver`

Créer une classe `DijkstraSolver` qui possède comme attribut une instance de `Graph` et qui propose une méthode permettant de calculer et de retourner le plus court chemin entre deux sommets du graphe. L'implémentation se basera sur l'algorithme de Dijkstra (les poids sont supposés tous positifs).

L'algorithme suit une stratégie gloutonne qui à chaque itération, sélectionne le chemin le plus court généré jusqu'à présent et le remplace par l'ensemble des chemins obtenus en étendant ce chemin à son voisinage direct (sans revenir en arrière).

Pour l'implémenter, on utilise généralement une structure de données qui permet de retrouver efficacement le plus petit élément (i.e. le plus court chemin) comme une `PriorityQueue`.

Le principe de l'algorithme est le suivant :

1. On ajoute à la file un chemin de poids 0 qui ne contient que le sommet source.
2. On récupère et on enlève le chemin le plus court de la file.
3. Si le sommet d'arrivée de ce chemin n'est pas la cible donnée, on le remplace par l'ensemble des chemins obtenus en étendant le chemin initial à son voisinage et

recommence à l'étape 2, sinon c'est fini.

Il faut aussi veiller à garder l'ensemble des sommets visités pour ne pas étendre un chemin vers un sommet déjà visité dans les itérations précédentes (c.-à-d. éviter les circuits). Pour cela, java fournit différentes implémentations d'ensemble : voir le type Set et ses implémentations concrètes.

Tester l'implémentation sur différents graphes exemples.

© 2025 by Jérôme David, CC BY-SA 4.0 